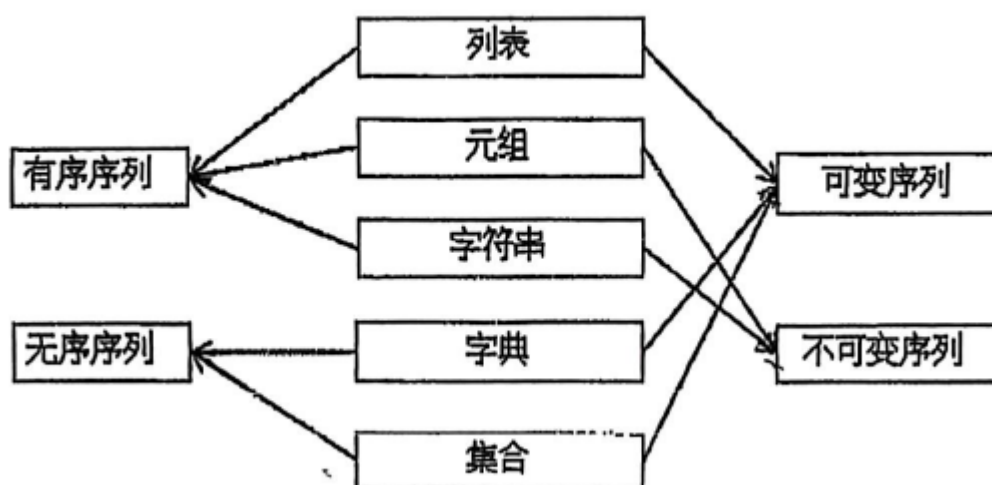


python学习笔记



列表

列表

方法	说明
<code>list.append(x)</code>	将元素x添加至列表尾部
<code>list.extend(L)</code>	将列表L中所有元素添加至列表尾部
<code>list.insert(index, x)</code>	在列表指定位置index处添加元素x
<code>list.remove(x)</code>	在列表中删除首次出现的指定元素
<code>list.pop([index])</code>	删除并返回列表对象指定位置的元素
<code>list.clear()</code>	删除列表中所有元素，但保留列表对象，该方法在Python2中没有
<code>list.index(x)</code>	返回值为x的首个元素的下标
<code>list.count(x)</code>	返回指定元素x在列表中的出现次数
<code>list.reverse()</code>	对列表元素进行原地逆序
<code>list.sort()</code>	对列表元素进行原地排序
<code>list copy()</code>	返回列表对象的浅拷贝，该方法在Python2中没有

列表的创建

1. 直接使用=创建

```
alist=[]#创建空列表
```

2. list(元组 or range or 字符串 等可迭代)

```
alist=list("hello world") #alist=['h','e','l','l','o',' ','w','o','r','l','d']
```

eg.创建含26个小写字母的集合：

```
s={chr(i+ord('a')) for i in range(26)}
```

列表的添加

1. 使用+

创建一个新列表，并将原列表中的元素和新元素依次复制到新列表的内存空间

```
alist=[1,2]  
alist=alist+[0]
```

2. 使用*

创建一个新列表

3. append(x)

原地修改列表，真正意义上在列表**尾部**添加**元素**

```
alist.append(0)#参数是元素
```

4. extend([x])

原地修改列表，将**另一个迭代对象**的所有元素添加至该列表对象**尾部**

```
alist.extend([0])#参数是列表
```

5. insert(index,value)

将元素添加至列表的**指定位置**。

列表的删除

基于索引

1. del

```
del a[0]
del a
```

2. pop(index)

删除**并返回**指定（默认为最后一个）位置上的元素

```
a=[1,2,4]
a.pop(1)
#返回2, a=[1,4]
```

基于值

1. remove(x)

删除首次出现的指定元素

```
a=[1,2,3]
a.remove(2)
```

遍历

```
a = [1,2,3,3]
for i in a[:]:#为了防止修改原列表，这里使用a[:]进行一个复制
...
```

列表的访问

1. index->value

```
a=[1,2,3]
a[0]
```

2. value->index

使用列表对象的 **index(value)** 方法获取指定元素首次出现的下标

```
#index(value)
a=[1,2,3]
a.index(3)#2
```

列表的判断

1. `x in alist`

2. `count(x)`

如果存在则返回大于0的数，如果不存在则返回0

列表的切片

1. 直接切片

切片操作不会因为下标越界而抛出异常

索引从0开始，左闭右开（同range）

```
a=[1,2,3,4,5]
b=a[1:]
#b=[2,3,4,5]
c=a[::-1]
#c=[5,4,3,2,1]
```

2. `del` 切片

```
del a[2:4:2]
```

列表的排序

本地排序

```
a=[1,2,3,4,5,6,7]
a.sort()
a.sort(reverse=True)
```

列表的BIF操作

列表的操作:BIF

len()	返回列表中的元素个数，同样适用于元组、字典、字符串等等
max()	返回列表中的最大元素，同样适用于元组、range。
min()	返回列表中的最小元素，同样适用于元组、range。
sum()	数值型列表元素的求和运算，对非数值型列表运算则出错
sorted()	对列表进行排序并返回新列表
reversed()	对列表元素进行逆序排列并返回迭代对象
zip()	将多个列表对应位置元素组合为元组，并返回这些元组的列表
enumerate()	枚举列表元素，返回每个元素的下标和值的元组

```
a=[0,1,2,3,4]
max(a)
#输出4
```

列表推导式

```
a=[i*i for i in range(5)]
#a=[0, 1, 4, 9, 16]
```

带条件判断的列表推导式

```
[i for i in a if i>0]
#[1, 4, 9, 16]
```

元组

元组中的数据一旦定义就不允许更改。

元组的创建

1. 直接= (a,b)

```
c=3,  
#c= (3, )
```

对多个变量同时赋值

```
x,y,z=1,2,3  
x,y,z  
#返回 (1, 2, 3)  
x  
#返回1
```

2. tuple()函数
里面放列表or字符串

字典

字典的创建

1. 直接使用{}

```
d1={'age': 19}
```

1. 使用dict()函数

```
d2=dict(age=19)
```

dict()函数比较严格，参数名必须是一个标识符，不能是纯数字，可以下划线开头

字典的操作

Method	Description
d.keys()	返回字典d中所有键的列表，类型为dict_keys。
d.values()	返回字典d中值的列表，类型为dict_values。
d.items()	返回字典d中由键和相应值组成的元组的列表，类型为dict_items。
d.clear()	删除字典d的所有条目。
d.copy()	返回字典d的浅复制拷贝，不复制嵌入结构。
d.update(x)	将字典x中的键值加入到字典d。
d.pop(k)	删除键值为k的键值对，返回k所对应的值。
d.get(k[,y])	返回键k对应的值，若未找到该键返回None。如果指定参数y的值，则未找到k时返回y的值。

字典的访问

<字典>[<键>]

```
d[key]
```

判断字典里面是否有某个键key

1. 使用in
2. 使用get()函数
d.get(key),返回字典d里面键为key的元素value

字典的添加&修改

1. 直接d[key]=value赋值

```
d1 = {1:'MON', 2:'TUE', 3:'WED', 4:'THU',5:'FRI', 6:'SAT'}  
d1[0] = 'SUN'
```

2. update
update函数参数是字典

```
d1.update({0:'Sun',1:'Mon'})
```

字典的删除

1. pop(key)
d.pop(key)删除指定key对应的元素**并返回**该元素的value。不存在会报KeyError的错。

字典推导式

1. 利用字典推导式实现字典键和值的对换。

前提：值没有相同的

```
d2={v:k for k,v in d.items()}
```

2. 利用字典推导式生成想要的字典

```
a=['a','b','c']
b=[1,2,3]
#method1
{a[i]:b[i] for i in range(len(a))}
#method2
{ch:b[i] for i,ch in enumerate(a)}
#method3
{ch:b[a.index(ch)] for ch in a}
#method4
{x:y for x,y in zip(a,b)}
```

集合

集合的特点：

1. 无序的，不能通过数字进行索引
2. 非重复的，不能出现重复的元素
python分为可变集合（set）和不可变集合（frozenset）

（可变）集合的创建

创建空集合只能使用set函数

1. set(字符串，列表or元组)函数

```
s1 = set('hello')
#s1为{'o', 'l', 'h', 'e'}，无序且自动去重
```

2. 直接={...}

```
s = {1,2,3,4}
```

eg.创建一个含26个小写字母的集合


```
s={chr(ord('a')+i) for i in range(26)}
```

集合的访问

只能循环遍历或使用in、not in来访问或判断集合元素

```
for i in s1:
```

集合的运算

Set-Operation

运算	描述	运算	描述
$x \in s$	检测x是否在集合s中	$x \text{ not in } s$	检测x是否不在集合s中
$s1 s2$	并集	$s1==s2$	判断集合是否相等
$s1\&s2$	交集	$s1\leq s2$	判断s1是否是s2的子集
$s1-s2$	差集	$s1<s2$	判断s1是否是s2的真子集
$s1^s2$	异或集，求s1与s2中相异元素	$s1\geq s2$	判断s1是否是s2的超集
$s1 =s2$	将s2的元素并入s1	$s1>s2$	判断s1是否是s2的真超集

$s1|=s2$ 同等与 $s1= s1|s2$

集合的操作

Set-Method

	方法	描述
	s1.union(s2)	s1 s2, 返回一个新的集合对象
	s1.difference(s2)	s1-s2, 返回一个新的集合对象
	s1.intersection(s2)	s1&s2, 返回一个新的集合对象
	s1.issubset(s2)	s1<=s2
	s1.issuperset(s2)	s1>=s2
*	s1.update(s2)	将s2的元素并入s1
*	s1.add (x)	增加元素x到s1
*	s1.remove(x)	从s1移除x, x不存在报错
*	s1.clear ()	清空s1
	s1.copy()	复制s1, 返回一个新的集合对象
	s1.union(s2)	s1 s2, 返回一个新的集合对象

前三个最好记记

字符串

字符串的格式化

格式字符

格式字符	说明
%s	字符串
%r	字符串
%c	单个字符
%b	二进制整数
%d	十进制整数
%i	十进制整数
%o	八进制整数
%x	十六进制整数
%e	指数（基底写为e）
%E	指数（基底写为E）
%f、%F、%F	浮点数
%g	指数(e)或浮点数（根据显示长度）
%G	指数(E)或浮点数（根据显示长度）
%%	字符"%"

1. format方法

str.format()

format方法返回的是格式化的字符串**副本**,不会改变字符串的值

```
{name}的计算机成绩是{cs}，{name}的数学成绩是{ms} '.format(cs=95,ms=92,name='y')
# 'y的计算机成绩是95，y的数学成绩是92 '
```

常用的字符串函数（ycc讲义版）

函数名	含 义	示 例
✓ find() 从左 rfind() 从右	find() 查找一个字符串在另一个字符串指定范围（默认是整个字符串）中首次出现的位置。rfind() 查找最后一次出现的位置，如果不存在则返回-1	s= "apple, peach, banana, peach" s.find("peach") 6 s.rfind("peach") 19 s.rfind("peach", 10, 25) 19 s.find("peach", 7, 20) -1
✓ count()	统计一个字符串在另一个字符串中出现的次数	s= "apple, peach, banana, peach" s.count("peach") 2 s.count("ppp") 0
endswith() startswith()	判断字符串是否以指定范围内的指定字符串开始或结束	s = 'Beautiful is better than ugly' s.startswith("Be") True Fname.endswith(('.bmp' , ' .jpg' , ' .gif'))
isalnum() isalpha() isdigit() isupper() islower()	用来测试字符串是否为数字或字母 是否为字母 是否为数字 用来测试字符串是否为大写字母 是否为小写字母	'1234abcd'.isalnum() True '1234abcd'.isalpha() False 'abcd'.isalpha() True '1234'.isdigit() True '1234.0'.isdigit() False
✓ lower() upper()	转换成小写字符串 转换成大写字符串	s= "What is Your Name?" s.lower() 'what is your name?' s.upper() 'WHAT IS YOUR NAME?'
replace()	类似于“查找与替换”功能	s="暴力，中国" s.replace("暴力", "***") ***，中国
✓ strip()	删除字符串两边的空白字符或指定字符	' abc '.strip() 'abc' '\n\nhello world \n\n'.strip() 'hello world' "aaaassddf".strip("a") 'ssddf'
✓ split()	用来以指定字符为分隔符，将字符串分割成多个字符串，并返回包含分割结果的列表。	①s="apple, peach, banana, pear" s.split(",") ['apple', 'peach', 'banana', 'pear'] ②s = "2014-10-31" list(map(int, s.split("-"))) [2014, 10, 31]
✓ join(iter)	不推荐使用+运算符连接字符串，优先使用 join() 方法	li=["apple", "peach", "banana", "pear"] ①"".join(li) 'apple peach banana pear' sl=['1', 'a', '2', 'b'] ②"-".join(sl) '1-a-2-b'